

Completion			
Load new completion system with:			
autoload -Uz compinit			
compinit			
Configuration: uses styles			
zstyle context style value...			
where context may be a pattern matching the following form:			
:completion:func:completer:cmd:arg:tag			
in which:			
completion			
Literal string always used by completion functions			
func			
Name of directly called widget, blank for contextual completion			
completer			
Method of completion e.g. complete ; see below			
cmd			
Name of command being completed, or special command context			
arg			
Only valid with standard parsing: arg-n for <i>n</i> th argument			
option-opt-n for <i>n</i> th argument of option opt			
tag			
Indication of type of thing to be completed at this point.			
Completers († indicates modifiers existing or later completions):			
†_all_matches	Later completers add all matches		
_approximate	Complete with errors in part so far		
_complete	Basic completion		
_correct	Correct word already typed		
_expand	Perform shell expansions		
_expand_alias	Expand aliases only		
_history	Complete words from shell history		
†_ignored	Reinstate matches omitted		
†_list	List on first completion, insert on second		
_match	Complete using patterns from line		
†_menu	Menu completion, no menu selection		
†_oldlist	Use existing list before generating new one		
_prefix	Complete ignoring what's after cursor		
Command contexts: any command name plus the special contexts:			
-array-value-	Element in array		
-brace-parameter-	Parameter within \${...}		
-assign-parameter-	Left hand side of assignment		
-command-	Word in command position		
-condition-	Word in [[...]] condition		
-default-	Word with no specific completion		
-equal-	Word beginning with equals sign		
-first-	Tried first, may set _compskip		
-math-	Inside arithmetic such as ((...))		
-parameter-	Parameter with bare \$ in front		
-redirect-	Word after redirection operator		
-subscript-	Inside parameter subscript		
-tilde-	Between ~ and first / of argument		
-value-	Right hand side of assignment		
Tags:			
accounts	For users-hosts style		
all-expansions	When expanding, everything at once		
all-files	All files rather than a subset		
arguments	Command arguments		
arrays	Names of array parameters		
association-keys	Keys of associative arrays		
bookmarks	Bookmarks for URLs, ZFTP, etc.		
builtins	Names of builtin commands		
characters	Character classes, stty characters		
colormapids	X colormap IDs		
colors	Names of colors, usually X		
commands	External commands, subcommands		
contexts	Contexts in zstyle		
corrections	Possible approximations, corrections		
cursors	X cursor names		
default	Nothing specific in certain contexts		
descriptions	Used in format style for matches		
devices	Device special files		
directories	Directories		
directory-stack	Entries in pushd directory stack		
displays	X displays		
domains	Network domain (DNS) names		
expansions	Individual expansions instead of all		
file-descriptors	Numbers of open file descriptors		
files	Generic file matching tag		
fonts	X font names		
fstypes	Files system types for mount etc.		
functions	Shell functions, possibly other types		
globbed-files	Names of files matched by pattern		
groups	UNIX groups		
history-words	Words from shell history		
hosts	Names of network hosts		
indexes	Indexes of arrays		
jobs	Shell jobs		
interfaces	Network interfaces (as from ifconfig)		
keymaps	ZLE keymaps		
keysyms	Names of X keysyms		
libraries	Names of system libraries		
limits	System resource limits		
local-directories	Subdirectories of current directories		
manuals	Names of manual pages		
mailboxes	E-mail folders		
maps	NIS maps etc.		
messages	Used in format style for messages		
modifiers	X modifiers		
modules	Shell modules etc.		
my-accounts	Own accounts, with users-hosts style		
named-directories	Directories named by a parameter		
names	Names of all sorts		
newsgroups	USENET newgroups		
nicknames	Nicknames of NIS maps		
options	Options to commands		
original	Original when correcting, expanding		
other-accounts	Other accounts with users-hosts style		
Tags continued:			
packages	RPM, Debian etc. packages		
parameters	Names of shell parameters		
path-directories	Directories under \$cdpath		
paths	Used with assorted directory paths		
pods	Perl documentation		
ports	TCP, UDP prots		
prefixes	URL etc. prefixes		
printers	Names of print queues		
processes	PIDs		
processes-names	Names of processes in killall		

sequences	MH sequences etc.
sessions	ZFTP sessions etc.
signals	System signal names, HUP etc.
strings	Assorted strings, e.g. second arg of cd
styles	Styles in zstyle
suffixes	Filename extensions
tags	Tags used with rpm etc.
targets	Targets inside Makefiles
time-zones	Time zones with TZ parameter etc.
types	Assorted types of anything
urls	Used with web addresses
users	Names of users
values	Values in lists
variant	Used when picking variant of command
visuals	X visuals
warnings	Used in the format style for warnings
widgets	Names of zsh widgets
windows	IDs of X windows
zsh-options	Shell options

Styles († indicates on by default):

accept-exact	Accept exact match even if ambiguous
†add-space	Add a space after expansions
ambiguous	Cursor after ambiguous path component
assign-list	PATH -style list on assignment
auto-description	String for option desc's without specific
avoid-completer	Avoid completer with <u>_all_matches</u>
cache-path	Path to top of various caches
cache-policy	Function to decide on cache rebuilding
call-command	If true, use external (slow) command
command	External command to call (+args)
command-path	Override PATH for commands to match
commands	Default sys init commands (start etc.)
complete	Complete aliases (<u>_expand_alias</u>)
completer	The list of completers to try (see above)
†condition	Delay insertion of matches (<u>_list</u>)
disabled	Disabled aliases (<u>_expand_alias</u>)
disable-stat	If set, _cvs uses ls instead of zsh/stat
domains	Net domains (/etc/resolv.conf)
expand	For prefix , suffix in multiple parts
fake	Add value:desc fake completions

fake-files	<i>dir:names</i> add <i>names</i> in <i>dir</i>
fake-parameters	Params to complete even if not yet set
file-patterns	<i>pattern:tag</i> generates files with tag
file-sort	size, links, time, access, inode, reverse
filter	In LDAP, attributes for filtering
force-list	Just list matches: always or number
format	Desc string, %d shows specific desc
†glob	Attempt glob expansion (<u>_expand</u>)
†global	Global aliases (<u>_expand_alias</u>)
group-name	Name groups shown together by tag
group-order	Order groups shown together by tag
groups	Unix groups, as per /etc/group
hidden	Complete but don't list matches
hosts	List of host names, as /etc/hosts
hosts-ports	List of <i>hosts:ports</i> for TCP/UDP
ignore-line	Don't complete words already present
ignore-parents	parent or pwd : ignore parent dirs
ignored-patterns	If pattern matched, don't complete
insert	All matches at once (<u>_all_matches</u>)
insert-ids	Convert %cmd to unambiguous PID
insert-tab	Insert TAB if no non-whitespace yet
insert-unambiguous	Only menu complete when no prefix to insert
keep-prefix	Try to keep expandable prefix
last-prompt	Return to last editing line if possible
list	Control listing when history completing
list-colors	Color specs like LS_COLORS
†list-grouped	Grouped listing shown more compactly
list-packed	All matches shown more compactly
list-prompt	Prompt when scrolling completions
list-rows-first	Increment rows first in lists
list-suffixes	Show ambiguous bits of multiple paths
list-separator	Separates description in verbose list
local	<i>host:path:dir</i> for URLs as files
mail-directory	Directory for mailbox files (~/Mail)
match-original	Add * when matching (<u>_match</u>)
matcher	Apply match control syntax per tag
matcher-list	Apply match control syntax globally
max-errors	Max errors allowed in approx/correct
max-matches-width	Cols to reserve for matches (not desc)
menu	Use menu completion

muttrc	Alternative for ~/muttrc
numbers	Prefer job numbers instead of name
old-list	Retain list of matches (<u>_oldlist</u>)
old-matches	Use old match list (<u>_all_matches</u>)
old-menu	Keep list for meus (<u>_oldlist</u>)
original	Add original match for approx/correct
packageset	For arguments of Debian dpkg
path	For X colors, path to rgb.txt
pine-directory	Directory for PINE mailboxes
ports	TCP/IP services (/etc/services)
prefix-hidden	Hide common prefix e.g. in options
prefix-needed	Common prefix must be typed by user
preserve-prefix	Initial file patterns to leave alone
range	Range of words in history to consider
†regular	Complete regular aliases

Styles continued:

†remote-access	Control remote access for e.g. _cvs
remove-all-dups	Never complete duplicates in history
select-prompt	Prompt shown in menu selection
select-scroll	Lines to scroll in menu selection
separate-sections	Manual sections used as part of tag
show-completer	Show progress of completers as msg
single-ignored	Control _ignore when single match
sort	Override sorting of matches
special-dirs	Add . and .. to file list
squeeze-slashes	fo//ba is fo/ba not fo/*/ba
stop	Pause before looping shell history
strip-comments	Remove display name from email addr
subst-globs-only	Only take expansions from globbing
†substitute	When expanding, first try subst
†suffix	Only expand path with no /suffix
tag-order	Preference order for tags in context
urls	Determine where URLs are taken from
use-cache	Control caching for various commands
use-compctl	Use compctl -style completions
use-perl	Use simpler Perl code for _make
users	List of user names
users-hosts	List of <i>user@host</i> possibilities
users-hosts-ports	List of <i>user@host:port</i>
†verbose	Verbose output e.g. option descriptions
word	Line changes based on current word

Using **_arguments** for parsing standard command arguments:
Three arguments give argument/option selector, message to output, action to take. Examples:

```
1:msg:_comp      First arg; show msg, exec _comp
1::msg:_comp     Same for optional argument
:msg:_comp       Arg number inferred from position
*:msg:_comp      Any of the remaining args ("rest args")
*:::msg:_comp    words etc. set to normal args
*::::msg:_comp   ... set to args for this chunk
-foo             Complete option -foo
+foo             Complete option +foo
+foo             Complete -foo or +foo
*-foo           Option may occur multiple times
-foo:esg:_comp   Option has arg in same word
-foo+:msg:_comp  Option has arg in same or next word
-foo=:msg:_comp  Option arg -foo=bar or -foo bar
-foo=:msg:_comp  Option arg is -foo=bar only
-foo[desc]       Option has description desc
*:pat:msg:_comp  Complete words up to pat
*:pat:msg:_comp  Modify words etc. for args
(-goo -boo)-foo  -foo excludes -goo, -boo
(*)-foo          -foo excludes rest args as matches
(:)-foo          -foo excludes normal args
(-)-foo          -foo excludes all options
!-foo            -foo should not be completed
*:msg:<space>     Show message but don't complete
*:msg:(a b)      Matches are listed items
*:msg:((a\dsc))  Matches with descriptions
*:msg:->string    Array state has string if matched
*:msg:{code}      Shell code generates matches
*:msg:= action    Insert dummy argument first
*:msg:_comp arg   Call _comp with additional args
*:msg: _comp arg  Call _comp with only given arg
-a - set1 -c - ... Common and specific completion sets
- "(set1)" -c - ... Mutually exclusive sets
-s               Allow combined single letters
-sw             Same, even if option has args
--              Guess options by using --help
-- -i pat       Same, ignoring options matching pat
```

Examples of other utility functions:

```
_alternative \
'users:user:_users' \
'hosts:host:_hosts'
```

Either users or hosts (tag, description, action)

```
_describe setdesc arr1 --
```

Associate descriptions with completions; *arr1* contains *completion:description* entries

```
_message text-msg
```

Don't complete, just output *text-msg*

```
_multi_parts sep array
```

Complete by parts with separator *sep*, *\$array* contains full matches.

```
_path_files
```

Complete files including partial paths; **_files** is smart front end; options **-f** all files (default), **-g pat** matching *pat* (with **_files** maybe directories too), **-/** directories only, **-W dirs** paths in which files are found, **-F files** files to ignore, overrides **ignored-patterns**

```
_sep_parts arr1 sep1 arr2 sep2 .....
```

Elements from *arr1*, then separator, then elements from *arr2*, etc.

```
_values -s sep desc spec1 spec2 ...
```

Complete multiple values separated by *sep*; values are given by *specs*, each of which is similar to **_arguments** option *spec* without leading -

```
_wanted thing expl 'my things' \
compadd mything1 mything2 ...
```

Typical way of adding completions **mything1** etc. with tag **things** and description **my things**; **expl** should be local variable. Use single tag, c.f. **_tags** and **_requested**

```
_tags tag1 tag2
```

```
_requested tag
```

Implement loops over different tags

```
_all_labels tag expl descr compcommand
```

```
_next_label tag expl descr
```

Implement loops over different labels for each **_requested** tag